KB금융그룹 | 국민의 평생 금융파트너

K-디지털 트레이닝

Vue 인스턴스

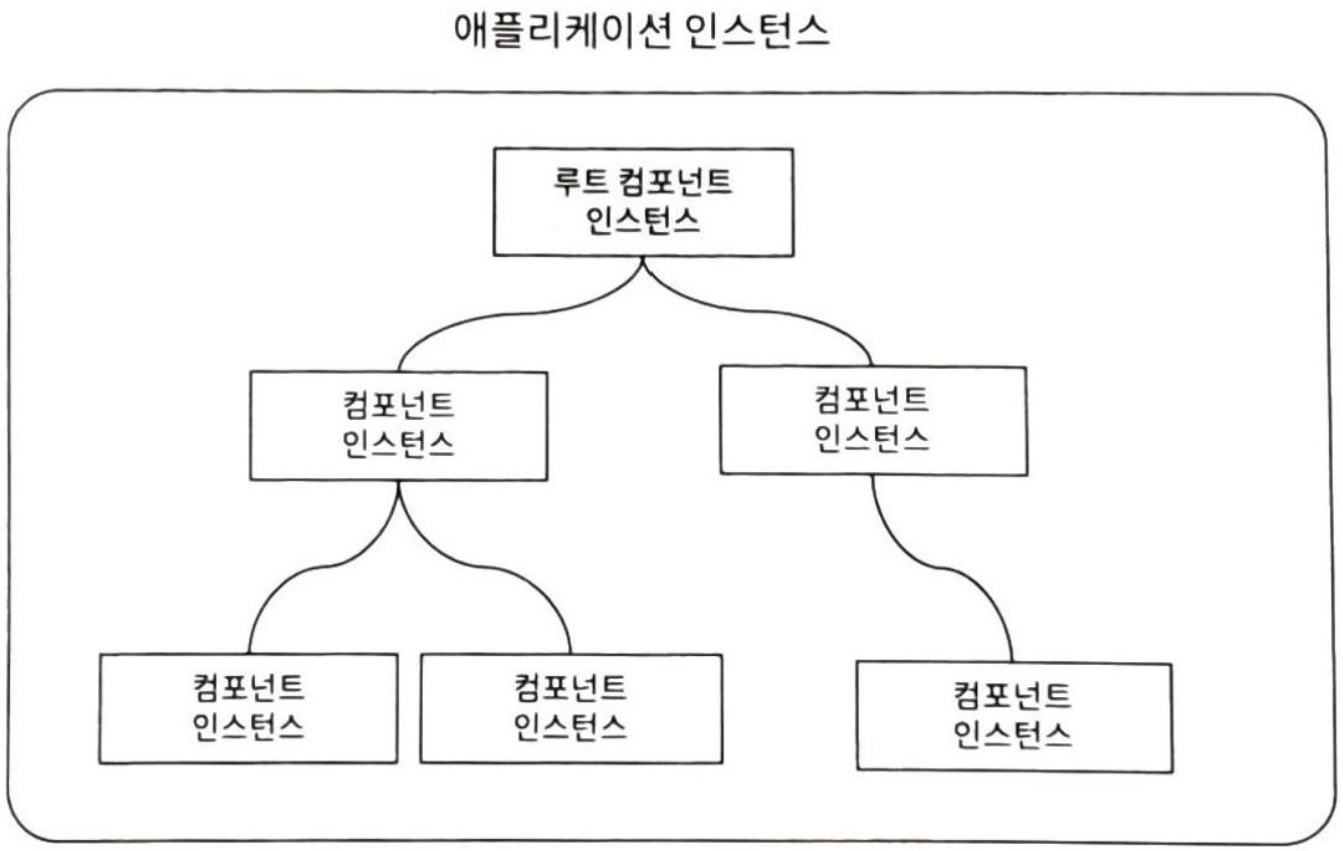
[KB] IT's Your Life

KB국민은행

애플리케이션 인스턴스

- Vue.createApp()로 만든 객체
- 전체 컴포넌트 트리는 애플리케이션 인스턴스를 마운트 할 때 렌더링하게 됨

```
let vm = Vue.createApp({
  name: "App",
  data() {
    return { name: ""};
  },
}).mount("#app");
```



data 옵션

- 컴포넌트가 관리하고 추적해야 할 데이터를 등록할 때 사용
  - 관리/추적 → 변경을 탐지하고 추적하여 화면을 갱신함

- 반드시 객체를 리턴하는 함수로 정의
- 리턴 객체가 Proxy로 래핑됨

vm.$data → data 옵션으로 구성된 proxy 객체

- 주의
  - $, _로 시작하는 이름은 Vue에서 특수한 용도로 사용하고 있음

  → 일반 식별자의 시작 문자로 사용하면 안됨

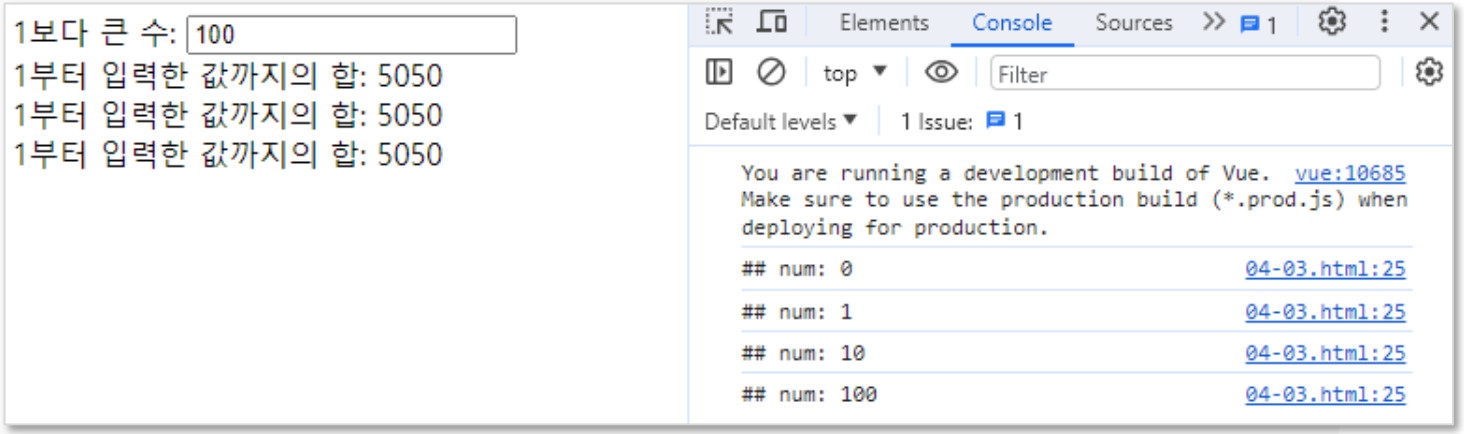
3 계산된 속성(Computed Property)

계산된 속성

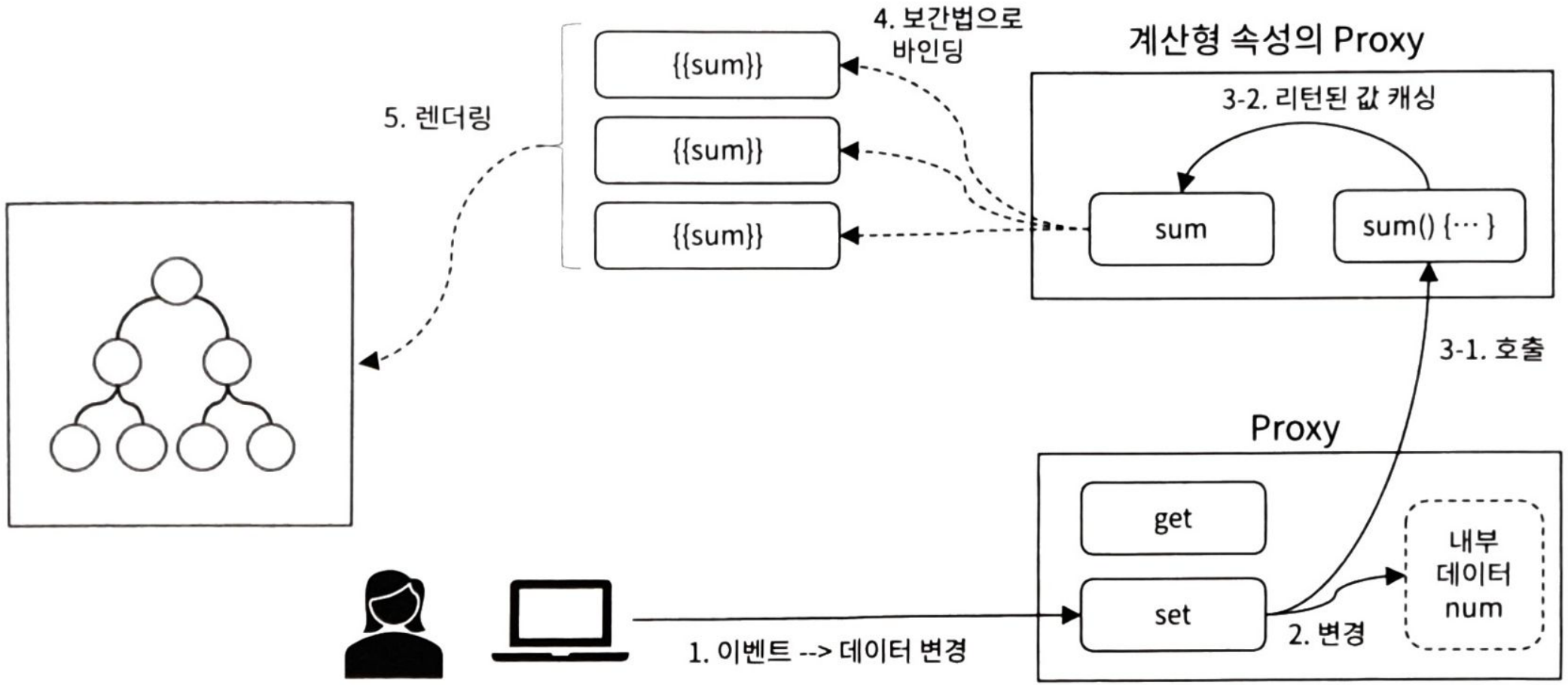
- data나 다른 속성이 변경될 때 함수가 실행되어 저장된 캐싱된 값
- 함수의 실행은 의존하고 있는 속성 또는 data가 변경될 때 한 번 호출됨
- computed 옵션에 함수를 등록
- 함수명이 계산된 속성명이 됨

04-03.html

```
<div id="app">
  1보다 큰 수: <input type="text" v-model.number="num" />
  <br />
  1부터 입력한 값까지의 합: <span>{{sum}}</span><br />
  1부터 입력한 값까지의 합: <span>{{sum}}</span><br />
  1부터 입력한 값까지의 합: <span>{{sum}}</span><br />
</div>
<script src="https://unpkg.com/vue"></script>
<script>
  let vm = Vue.createApp({
    name: "App",
    data() {
      return { num: 0 };
    },
    computed: {
      sum() {
        console.log("## num: " + this.num);
        let n = parseInt(this.num);
        if (Number.isNaN(n)) return 0;
        return (n * (n + 1)) / 2;
      },
    },
  }).mount("#app");
</script>
```



계산된 속성(Computed Property)의 작동 방식

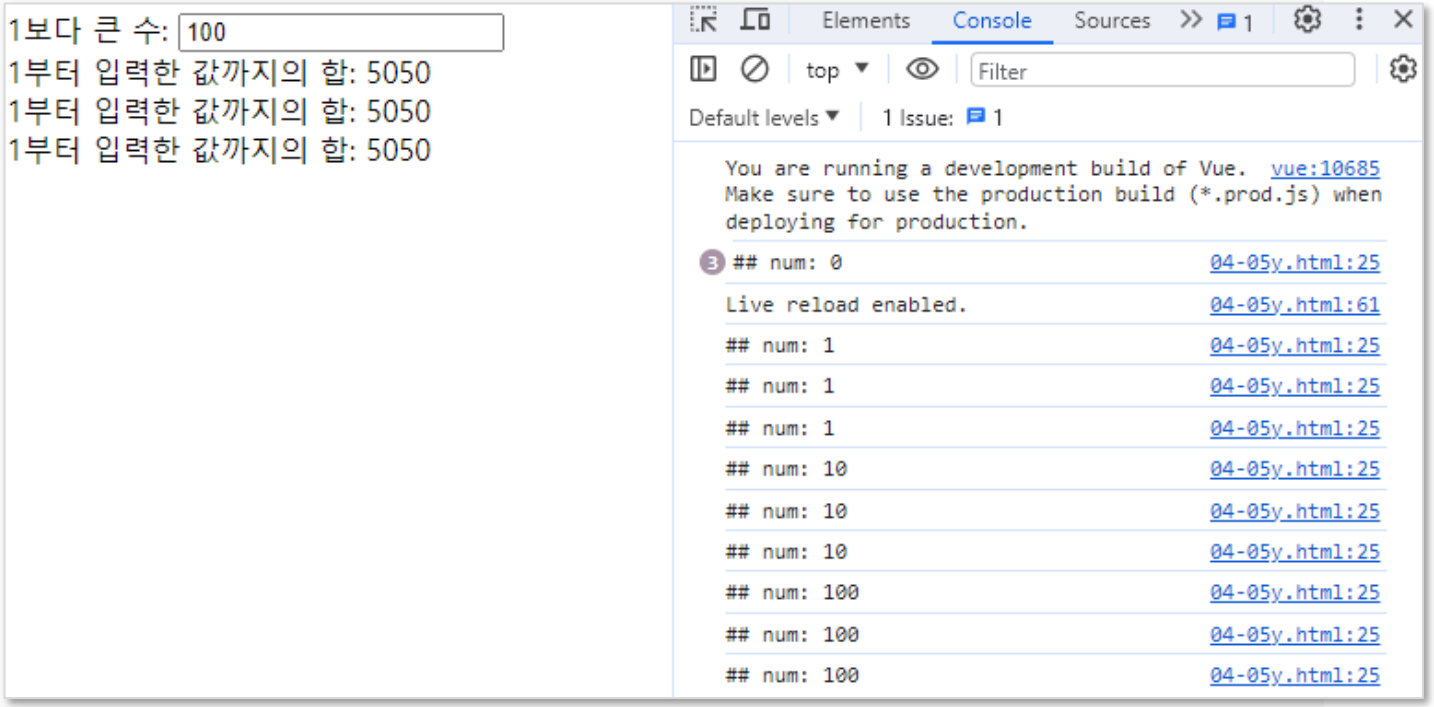


메서드 옵션

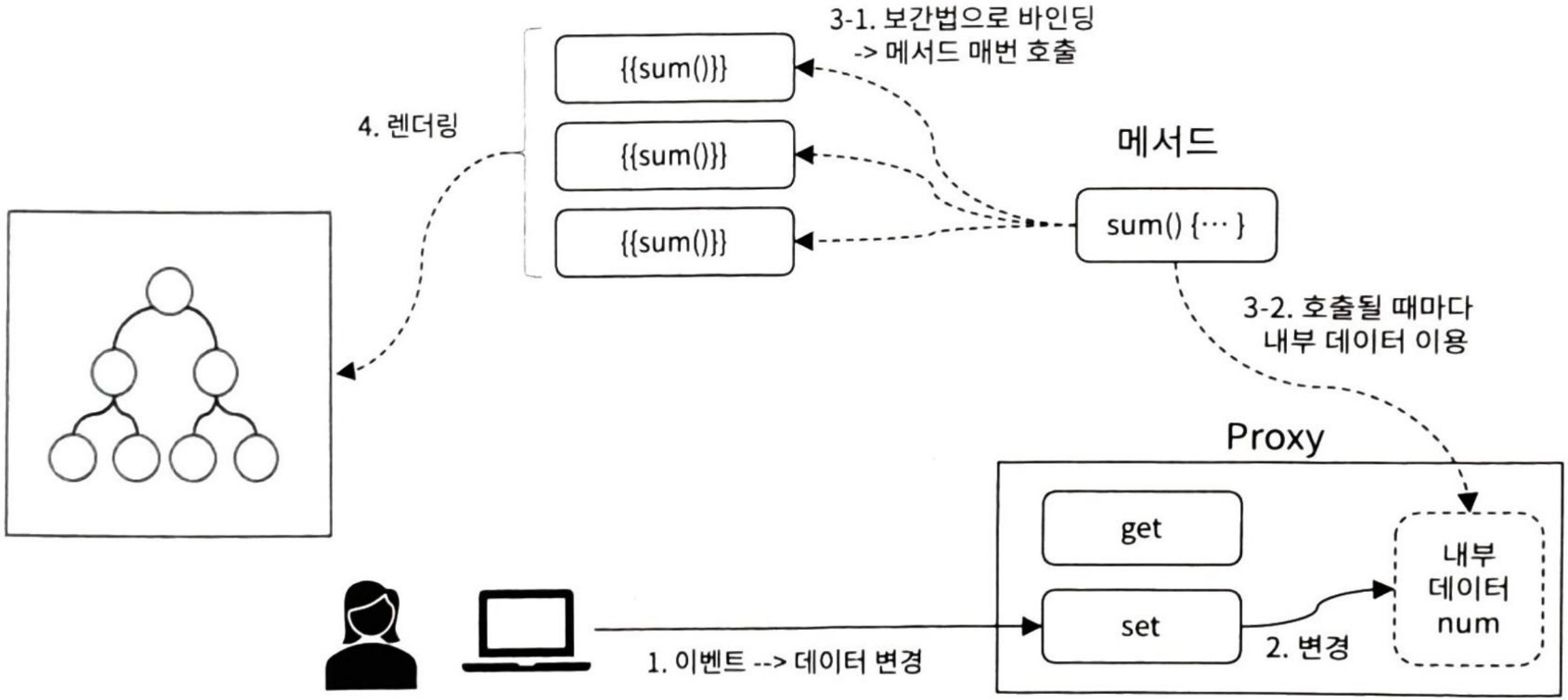
- Vue 인스턴스에서 사용할 메서드를 등록하는 옵션
- 인스턴스에서 직접 호출하거나, 디렉티브의 표현식, {{}}의 표현식에서 호출 가능
- 이벤트 핸들러로 이용가능

04-05.html

```
<div id="app">
  1보다 큰 수: <input type="text" v-model.number="num" />
  <br />
  1부터 입력한 값까지의 합: <span>{{sum()}}</span><br />
  1부터 입력한 값까지의 합: <span>{{sum()}}</span><br />
  1부터 입력한 값까지의 합: <span>{{sum()}}</span><br />
</div>
<script src="https://unpkg.com/vue"></script>
<script>
  let vm = Vue.createApp({
    name: "App",
    data() {
      return { num: 0 };
    },
    methods: {
      sum() {
        console.log("## num: " + this.num);
        let n = parseInt(this.num);
        if (Number.isNaN(n)) return 0;
        return (n * (n + 1)) / 2;
      },
    },
  }).mount("#app");
</script>
```



methods 옵션의 실행 방식



관찰 속성 Watched Property

JS 코드에서 데이터가 변경된 경우

- 이를 감지하여 다른 JS 코드를 실행하고자 하는 경우
- 주로 긴 처리 시간이 필요한 비동기 처리에 적합

watch 옵션 객체로 정의

- 함수로 정의
- data 속성명과 동일하게 함수명 정의
- 매개변수
  - 첫 번째 인자: 변경 후의 값
  - 두 번째 인자: 변경 전의 값

data 또는 속성명 변경후 x값 변경전 x값

```
x ( current , old ) {
    console.log(`## x : ${old} --> ${current}`)
    var result = Number(current) + Number(this.y);
    if (isNaN(result)) this.sum = 0;
    else this.sum = result;
}
```

04-06.html watch 옵션

```
<div id="app">
  x : <input type="text" v-model.number="x" /><br />
  y : <input type="text" v-model.number="y" /><br />
  덧셈 결과 : {{sum}}
</div>

<script type="text/javascript" src="https://unpkg.com/vue"></script>
<script type="text/javascript">
  var vm = Vue.createApp({
    name: "App",
    data() {
      return { x: 0, y: 0, sum: 0 };
    },
```

04-06.html watch 옵션

```
    watch: {
      x(current, old) {
        console.log(`## x : ${old} --> ${current}`);
        var result = Number(current) + Number(this.y);
        if (isNaN(result)) this.sum = 0;
        else this.sum = result;
      },
      y(current, old) {
        console.log(`## y : ${old} --> ${current}`);
        var result = Number(this.x) + Number(current);
        if (isNaN(result)) this.sum = 0;
        else this.sum = result;
      },
    },
  }).mount("#app");
</script>
```

04-07.html 계산된 속성과의 비교

```
<div id="app">
  x : <input type="text" v-model.number="x" /><br />
  y : <input type="text" v-model.number="y" /><br />
  덧셈 결과 : {{sum}}
</div>
<script type="text/javascript" src="https://unpkg.com/vue"></script>
<script type="text/javascript">
  var vm = Vue.createApp({
    name: "App",
    data() {
      return { x: 0, y: 0 };
    },
    computed: {
      sum() {
        let result = Number(this.x) + Number(this.y);
        if (isNaN(result)) return 0;
        else return result;
      },
    },
  }).mount("#app");
</script>
```

04-08.html 비동기 처리 관찰

```
<div id="app">
  이름 : <input type="text" v-model.trim="name" placeholder="영문 두글자 이상을 입력하세요" /><br />
  <ul>
    <li v-for="c in contacts">{{c.name}} : {{c.tel}}</li>
  </ul>
  <div v-show="isLoading">검색중</div>
</div>
<script type="text/javascript" src="https://unpkg.com/vue"></script>
<script type="text/javascript" src="https://unpkg.com/axios"></script>
<script type="text/javascript" src="https://unpkg.com/lodash"></script>
<script type="text/javascript">
  const BASEURL = "https://contactsvc.bmaster.kro.kr";
  var vm = Vue.createApp({
    name: "App",
    data() {
      return { name: "", contacts: [], isLoading: false };
    },
```

04-08.html 비동기 처리 관찰

```
    watch: {
      name(current) {
        if (current.length >= 2) {
          this.fetchContacts();
        } else {
          this.contacts = [];
        }
      },
    },
    methods: {
      fetchContacts: _.debounce(function () {
        this.isLoading = true;
        axios.get(BASEURL + `/contacts_long/search/${this.name}`).then((response) => {
          this.isLoading = false;
          this.contacts = response.data;
        });
      }, 300),
    },
  }).mount("#app");
</script>
```

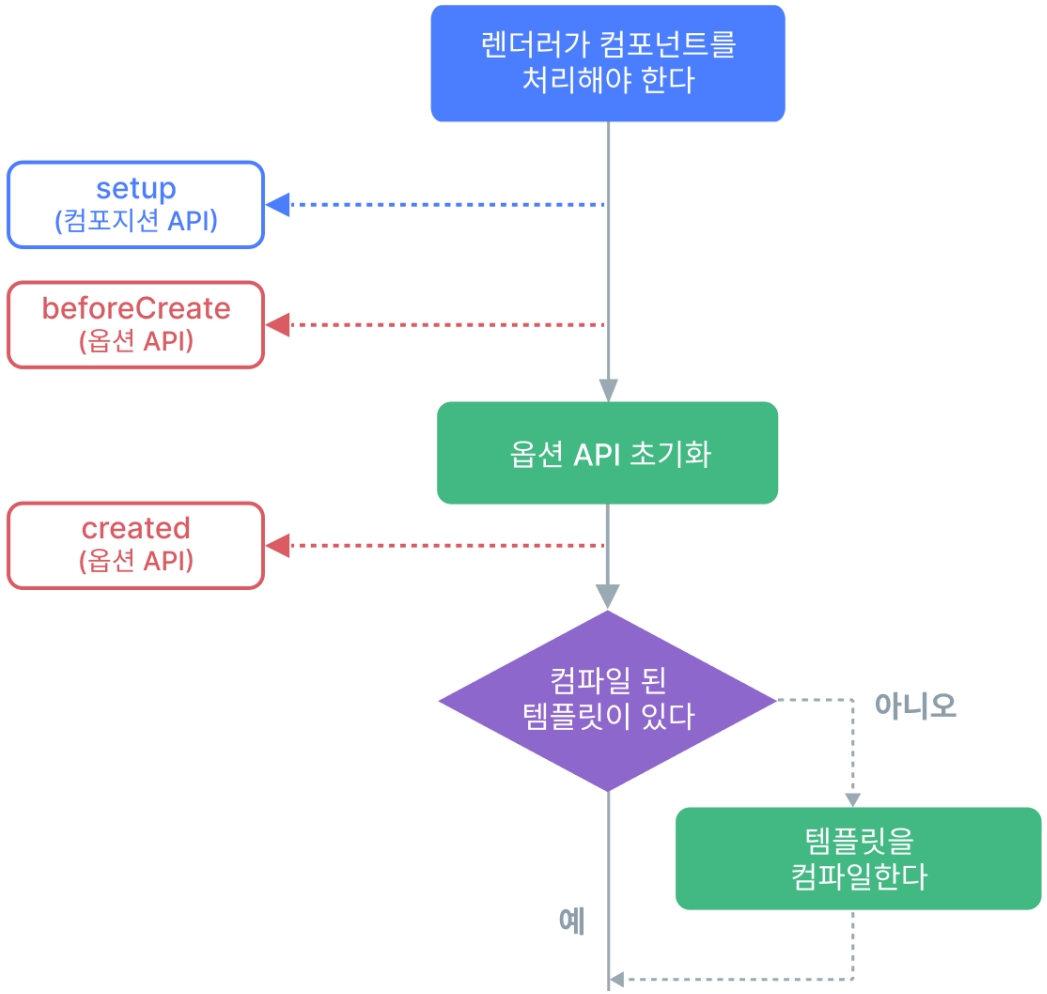
04-09.html 비동기 처리 관찰

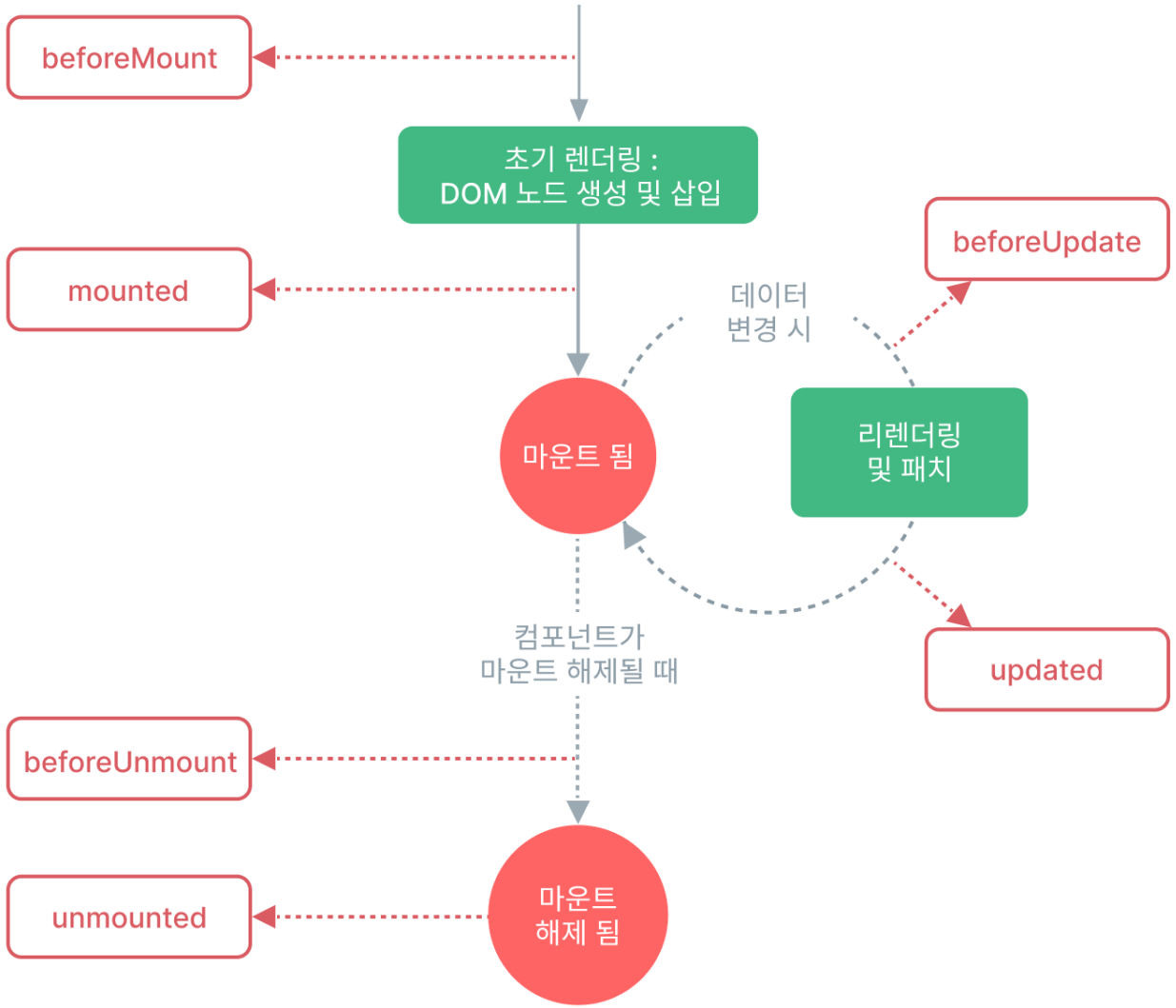
```
<div id="app">
  x : <input type="text" v-model.number="values.x" /><br />
  y : <input type="text" v-model.number="values.y" /><br />
  덧셈 결과 : {{sum}}
</div>
<script type="text/javascript" src="https://unpkg.com/vue"></script>
<script type="text/javascript">
  var vm = Vue.createApp({
    name: "App",
    data() {
      return { values: { x: 0, y: 0 }, sum: 0 };
    },
    watch: {
      values(current) {
        var result = Number(current.x) + Number(current.y);
        if (isNaN(result)) this.sum = 0;
        else this.sum = result;
      },
    },
  }).mount("#app");
</script>
```

04-09.html 수정: 비동기 처리 관찰

```
…
<script type="text/javascript" src="https://unpkg.com/vue"></script>
<script type="text/javascript">
  var vm = Vue.createApp({
    name: "App",
    data() {
      return { values: { x: 0, y: 0 }, sum: 0 };
    },
    watch: {
      values: {
        handler: function (current) {
          var result = Number(current.x) + Number(current.y);
          if (isNaN(result)) this.sum = 0;
          else this.sum = result;
        },
        deep: true,
      },
    },
  }).mount("#app");
</script>
```

Vue 생명주기


렌더러가 컴포넌트를
처리해야 한다
setup
(컴포지션 API)
beforeCreate
(옵션 API)
옵션 API 초기화
created
(옵션 API)
컴파일 된
템플릿이 있다
아니오
템플릿을
컴파일한다
예


beforeMount
초기 렌더링 :
DOM 노드 생성 및 삽입
mounted
beforeUpdate
데이터
변경 시
마운트 됨
리렌더링
및 패치
updated
컴포넌트가
마운트 해제될 때
beforeUnmount
unmounted
마운트
해제 됨

컴포넌트 생명주기 메서드

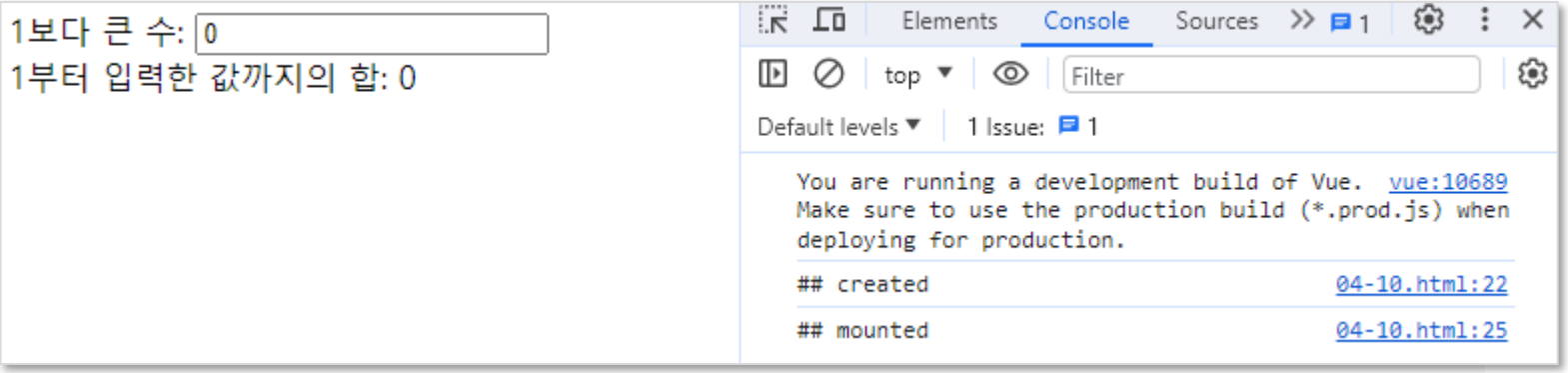
생명주기 메서드	설명
beforeCreate	인스턴스가 초기화된 후, 데이터에 대한 관찰 기능, 이벤트가 설정되기 전에 동기적으로 호출
created	인스턴스 생성이 완료된 후 동기적으로 호출됨. 데이터 관찰 기능, 계산된 속성, 메서드, 이벤트 초기화 등이 모두 완료된 상태임
beforeMount	마운팅이 시작되기 직전에 호출됨.
mounted	mount()에 의해 컴포넌트 인스턴스가 HTML Document의 요소에 마운트된 후에 호출
beforeUpdate	데이터가 변경되고 DOM이 패치되기 전에 호출됨.
updated	데이터의 변경으로 가상 DOM이 다시 렌더링되고 패치된 후에 호출됨 컴포넌트의 DOM이 업데이트된 상태로 DOM에 종속성이 있는 처리를 이 단계에서 수행
beforeUnmount	컴포넌트 인스턴스가 언마운트되기 직전에 호출됨
unmounted	컴포넌트이 인스턴스가 언마운트된 후에 호출됨(자식도 모두 언마운트된 상태임) 컴포넌트의 모든 디렉티브들의 연결이 해제되고, 모든 이벤트 리스너가 제거됨
errorCaptured	컴포넌트의 자식 컴포넌트 트리에서 오류가 발생하면 컴포넌트 트리를 거슬러 올라와서 이 메서드가 실행됨 컴포넌트 자신에서 발생한 오류는 잡아내지 못함

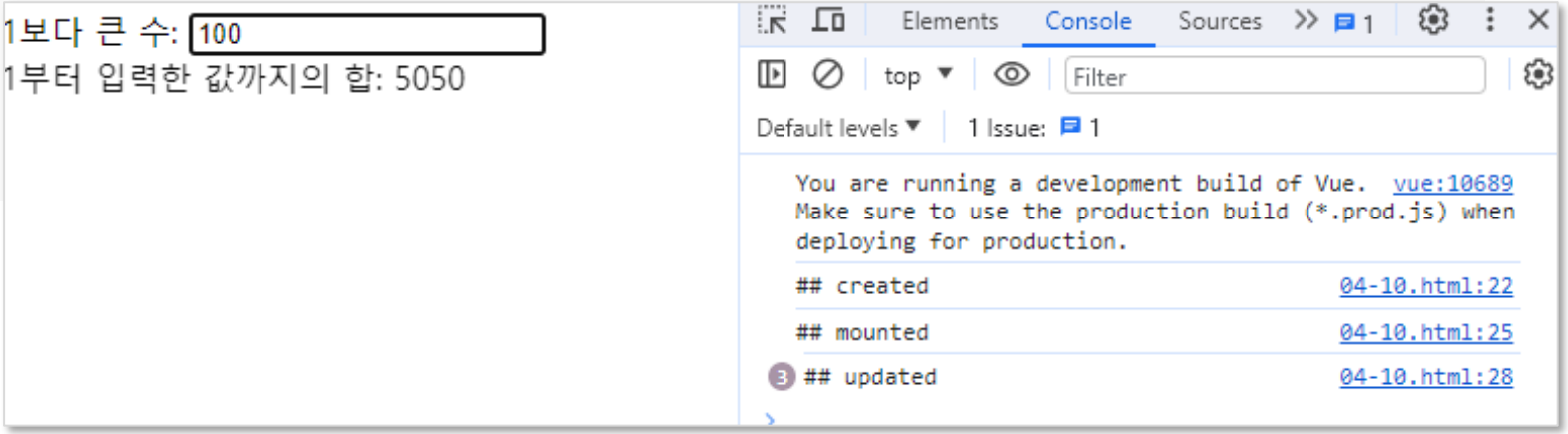
04-10.html

```
<div id="app">
  1보다 큰 수: <input type="text" v-model.number="num" />
  <br />
  1부터 입력한 값까지의 합: <span>{{sum}}</span><br />
</div>

<script src="https://unpkg.com/vue"></script>
<script>
  let vm = Vue.createApp({
    name: "App",
    data() {
      return { num: 0 };
    },
    computed: {
      sum() {
        let n = parseInt(this.num);
        if (Number.isNaN(n)) return 0;
        return (n * (n + 1)) / 2;
      },
    },
```

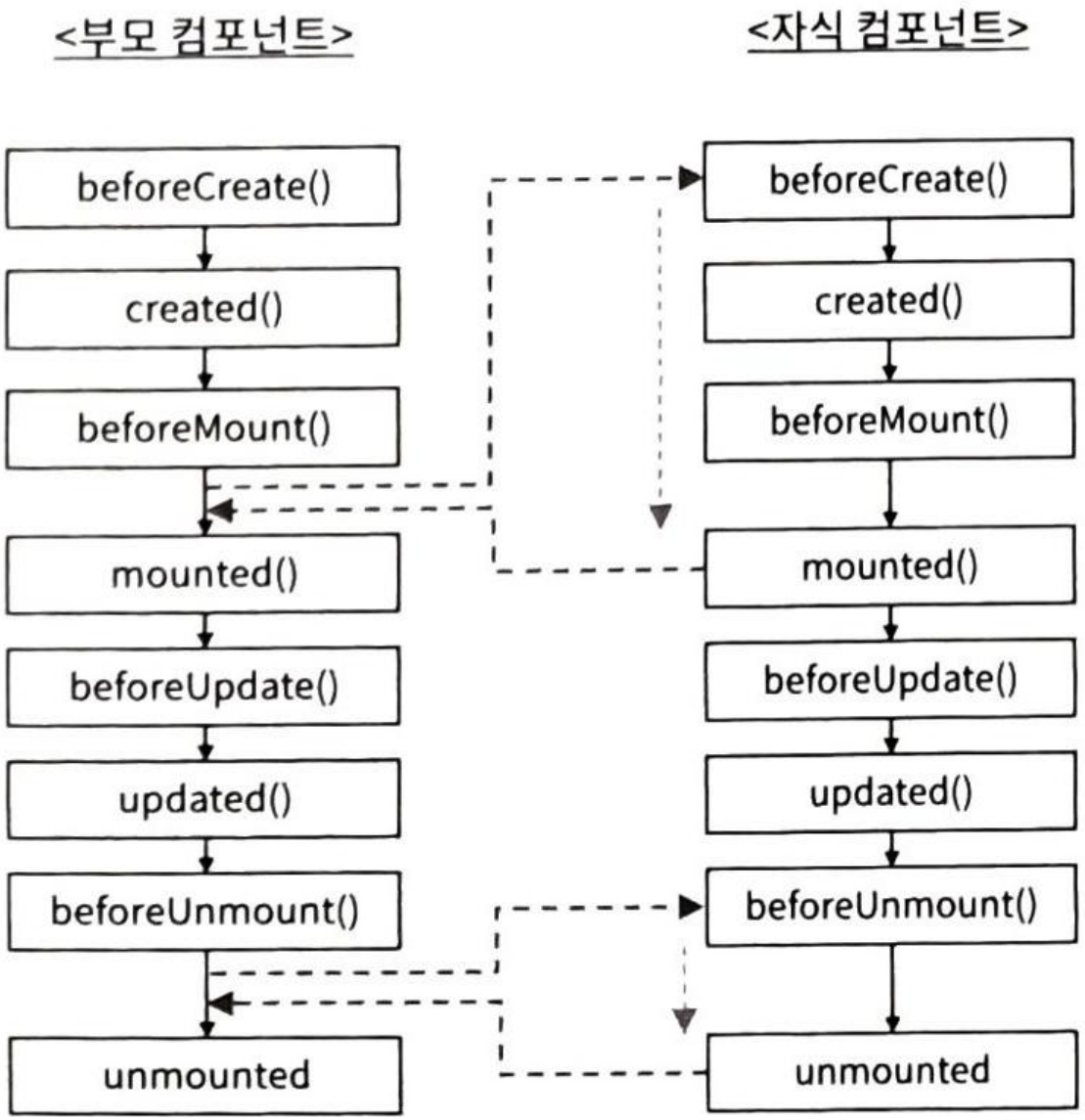
04-10.html

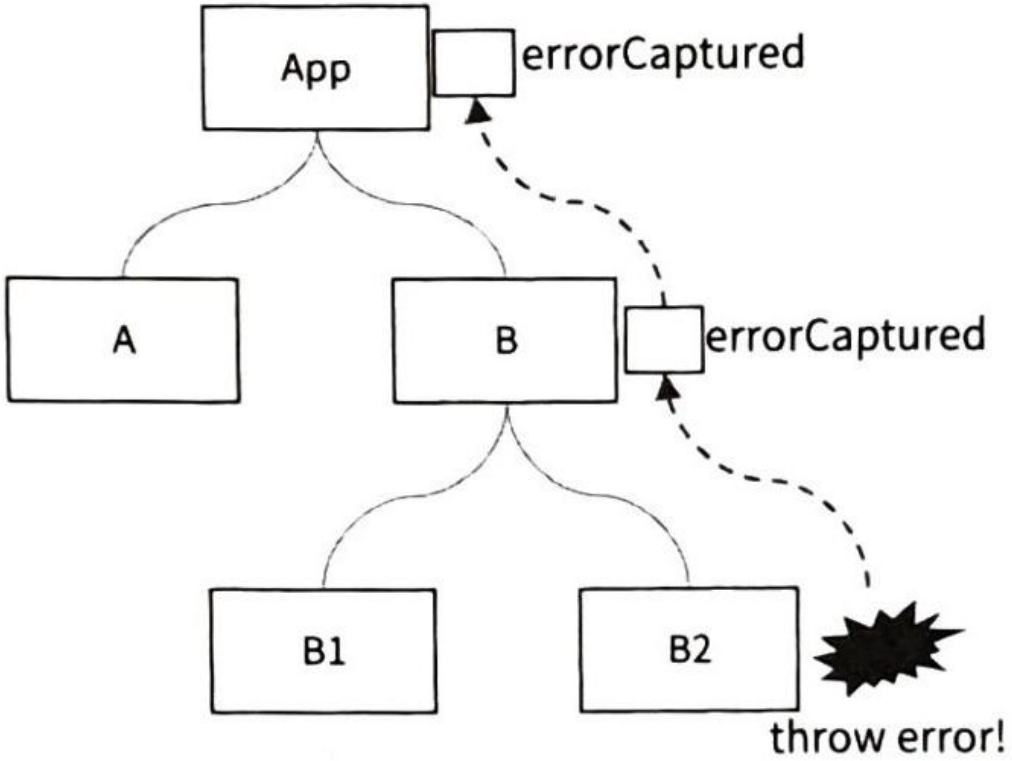
```
    created() {
      console.log("## created");
    },

    mounted() {
      console.log("## mounted");
    },

    updated() {
      console.log("## updated");
    },

  }).mount("#app");
</script>
```





부모-자식 컴포넌트 관계의 실행 흐름



컴포넌트 트리에서의 에러 처리

- 프로젝트 수준이나 주요 모듈 단위로 정의해서 중앙 집중화된 에러 처리를 수행함